

Optimizing Parallel Processes

Parallel and Heterogeneous Computing

David Mayerich

Scalable Tissue Imaging and Modeling (STIM) Laboratory

Department of Electrical and Computer Engineering

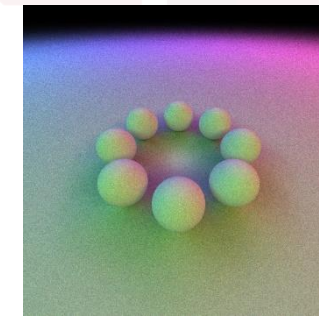
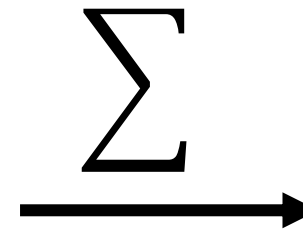
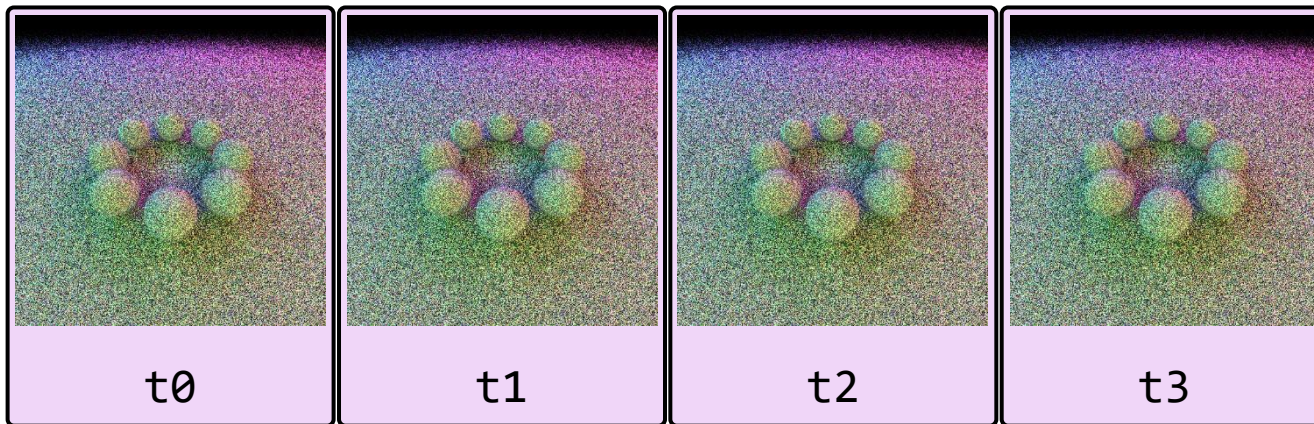
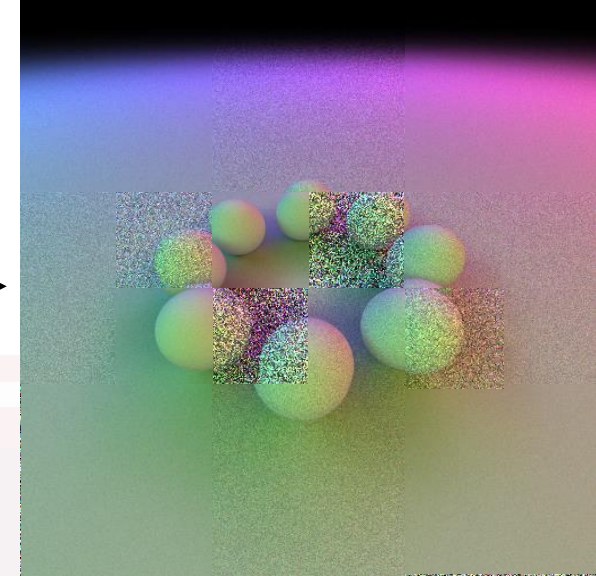
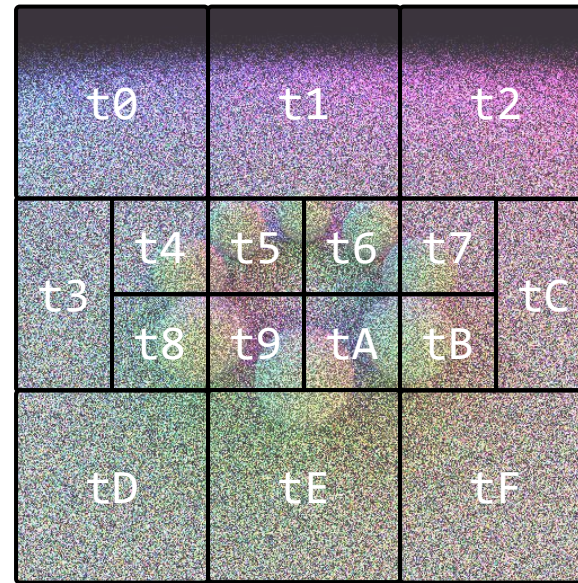
Cullen College of Engineering

University of Houston

UNIVERSITY of **HOUSTON** | ECE

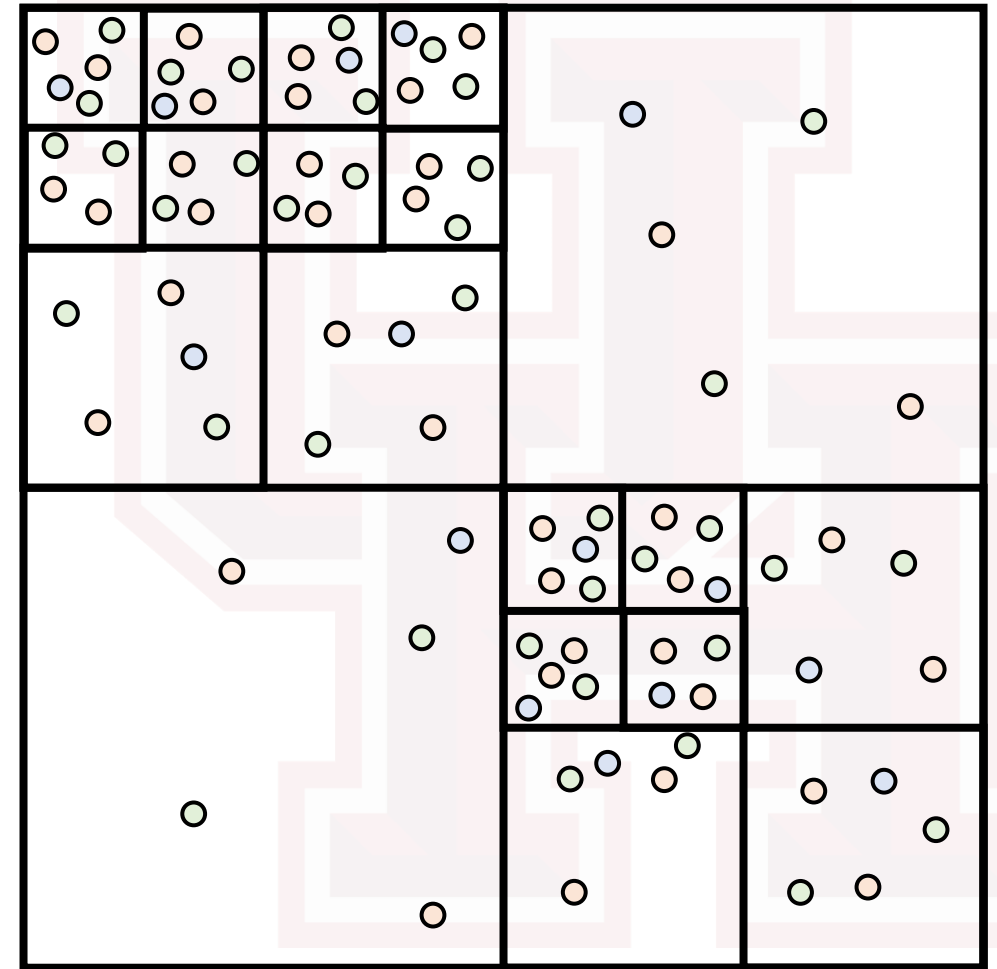
Static Assignment

- Allocate work before assigning it
- Specify rules at compile-time
- Assignment remains constant
- When is this useful?
 - Work is predictable
 - Work is *statistically* predictable



Semi-Static Assignment

- Determine the cost of work in the near term
- Assign workers based on that assumed cost
- Ex. Lagrangian particle simulation
- Ideas?
 - Short-term profiling: evaluate load per worker and re-distribute
 - Not all threads will be equally efficient – some cores may be busy with external tasks

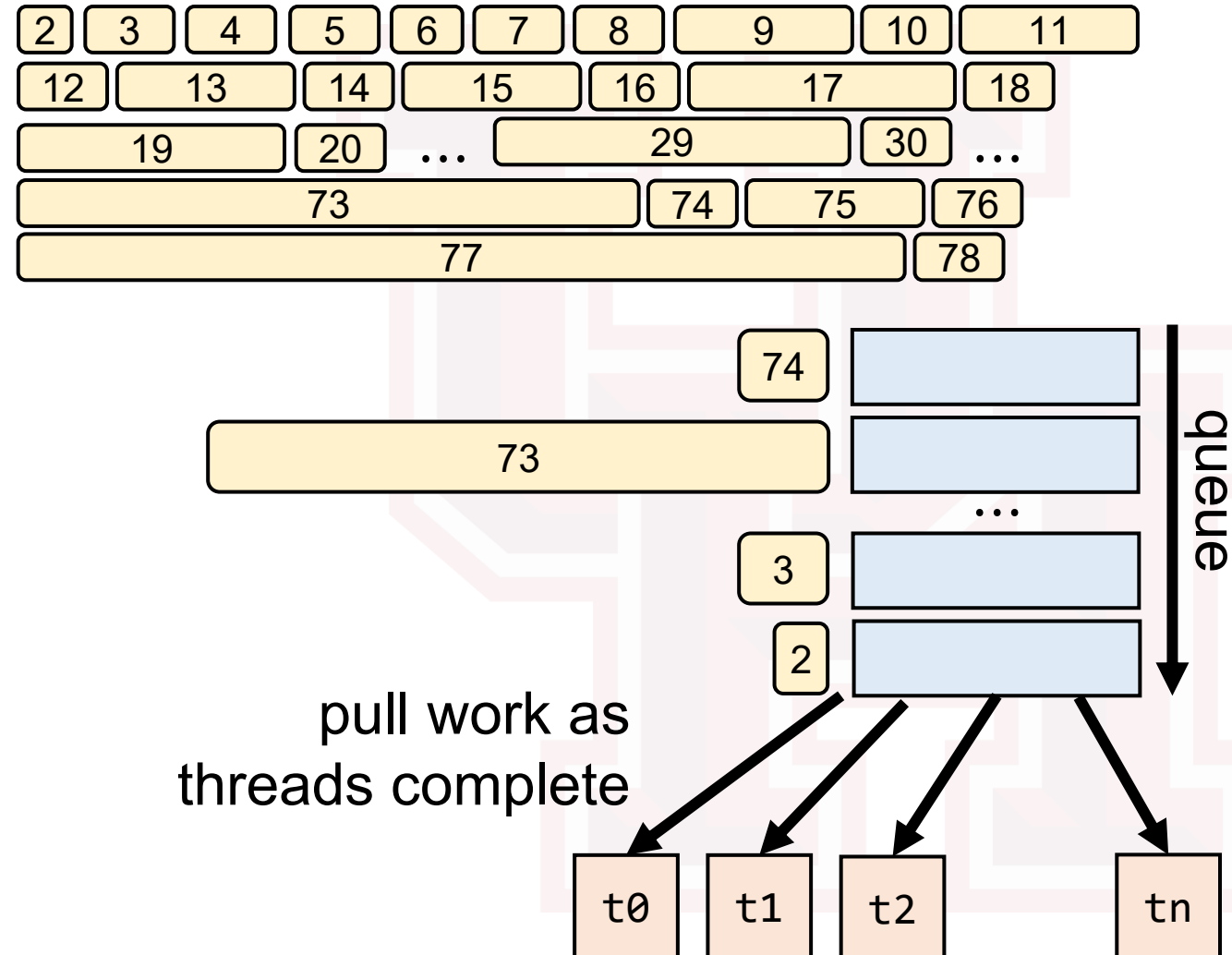


Dynamic Assignment

- Test for prime numbers:

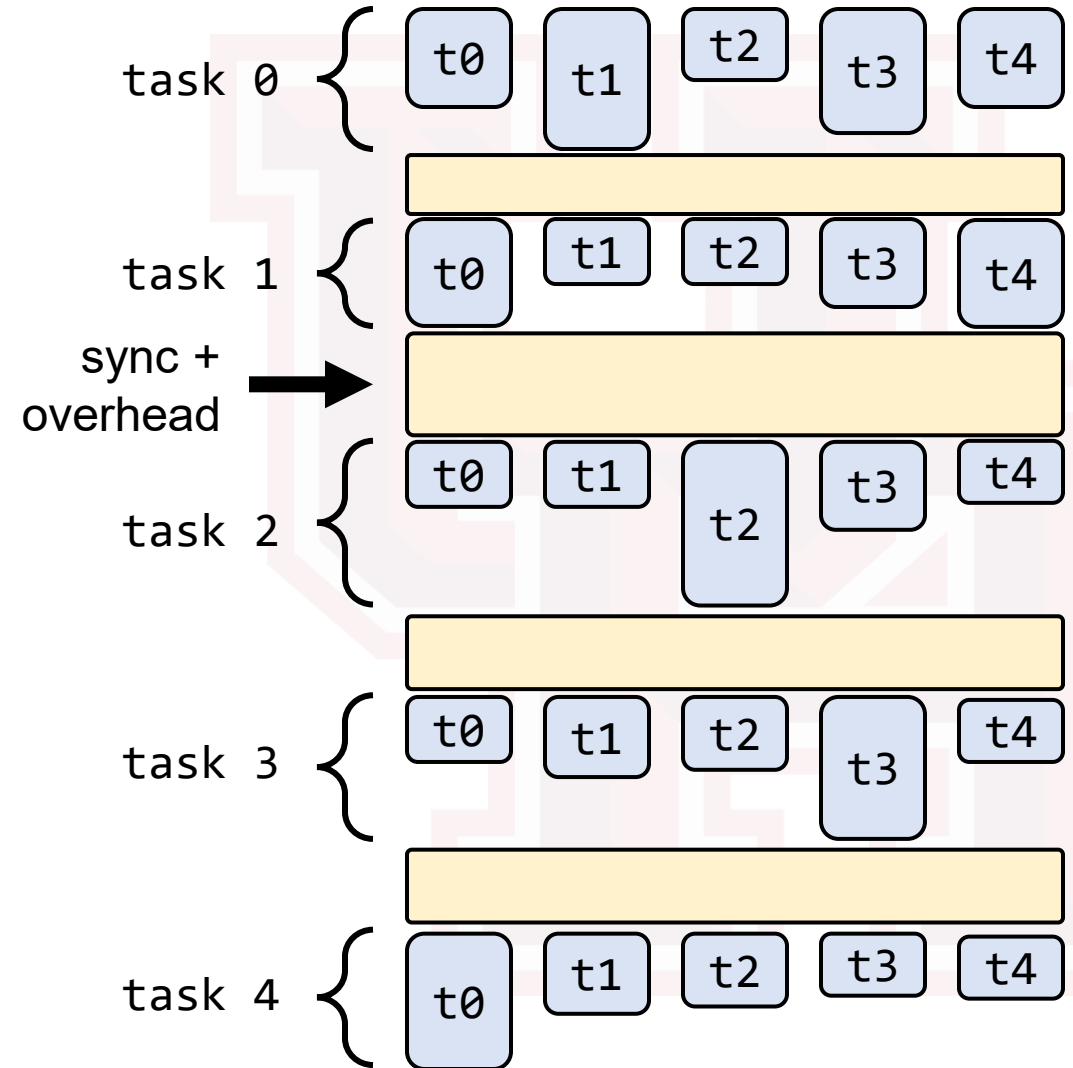
```
for(size_t i = 0; i < N; i++) {  
    p[i] = test_prime(x[i]);  
}
```

```
// test prime using trial division  
bool test_prime(int y) {  
    int m = sqrt(y);  
    for(size_t j = 2; j < m; j++) {  
        if(y % j == 0) return false;  
    }  
    return true;  
}
```



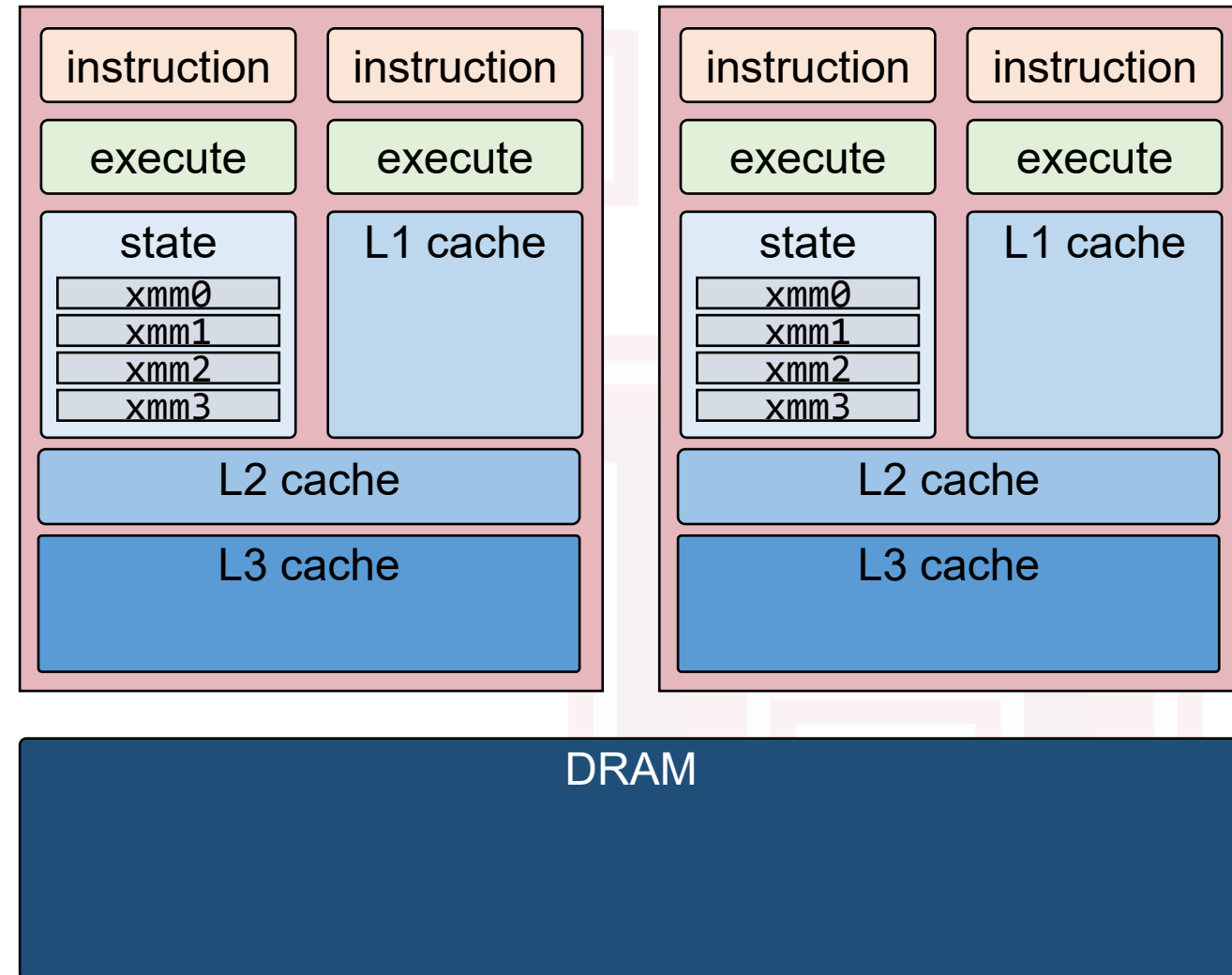
Assess Assignment Strategies

- Assignment strategies introduce overhead
 - synchronization: pause execution to wait for dependent threads
 - calculate assignment
 - memory latency
- How can you assess what's best?
 - Profile the entire program
 - Time total execution
 - Time total computation
 - $\text{execution} - \text{compute} = \text{overhead}$



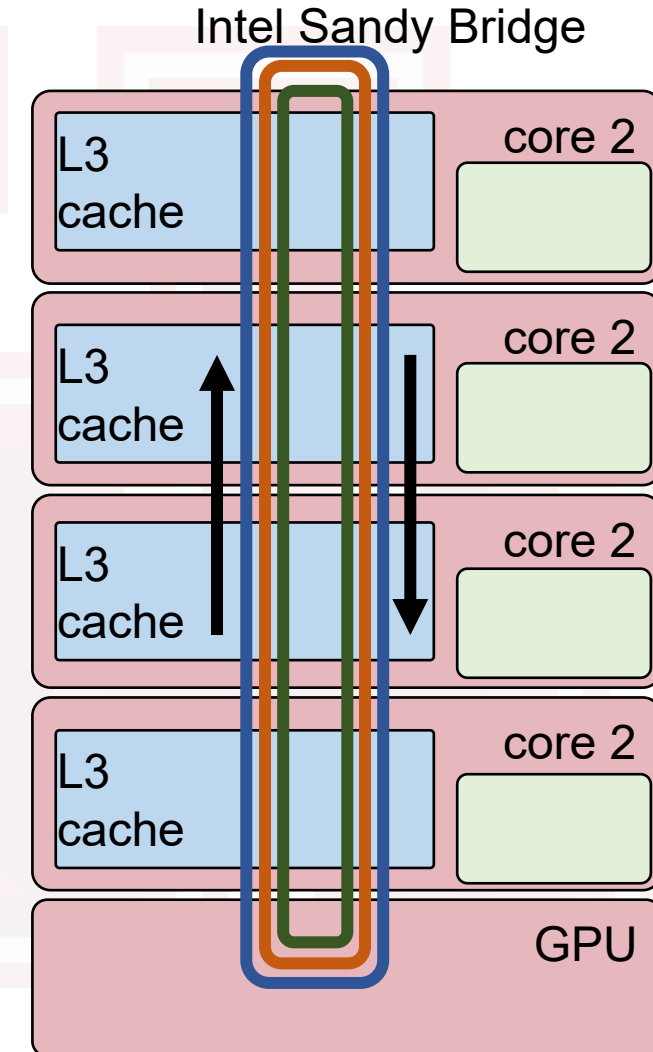
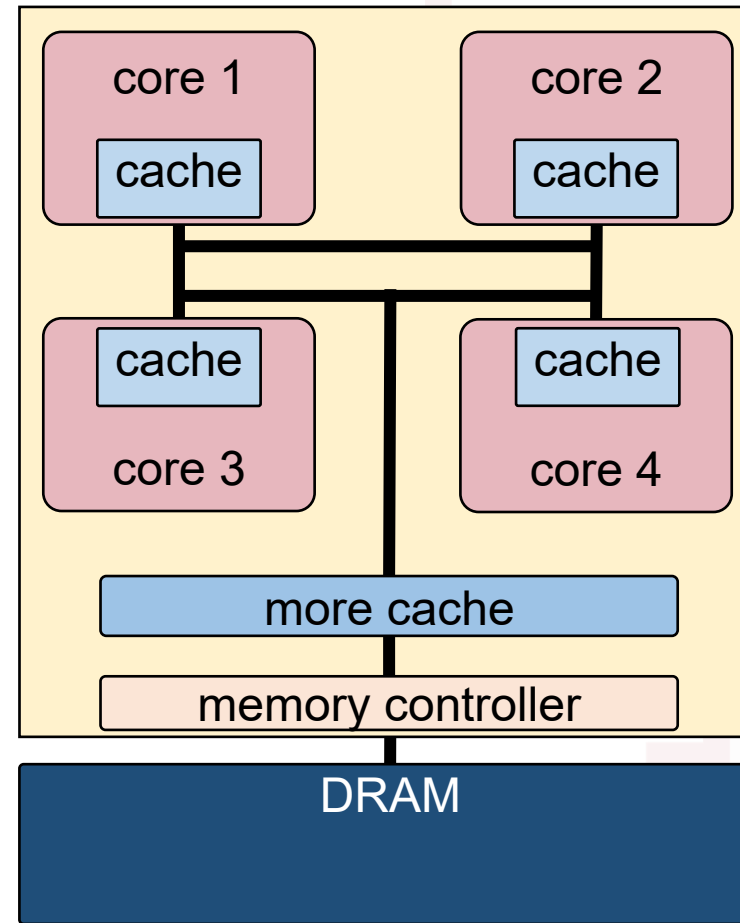
Shared Memory Address Spaces

- Conceptually simple at a high level
- Architecture obfuscates complexity
- Example
 - P0 and P1 fetch nearby values in DRAM
 - Cached values are stored in L3, L2, and L1 in both processors
 - Writes are made to values within this cache line
- What values are valid?
- Does the programmer worry about this?



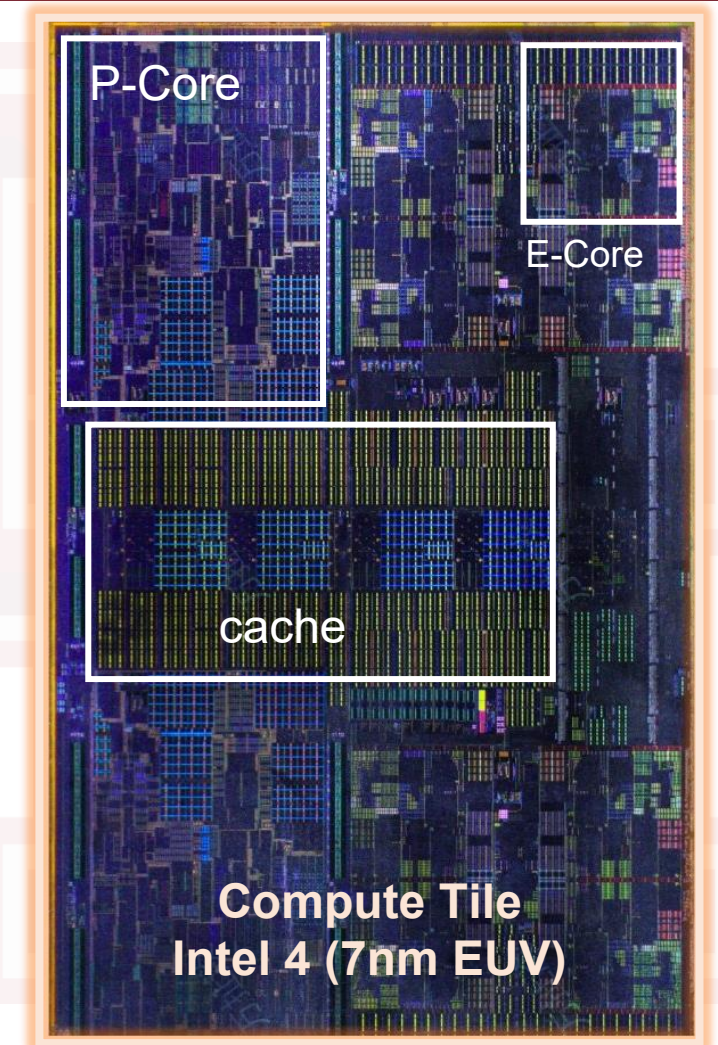
Memory Architectures

- Memory architectures form networks
- Networks have control units
- Caches are checked and synchronized
- Ring architecture:
 request snoop data
- Why 2 connections per cache?



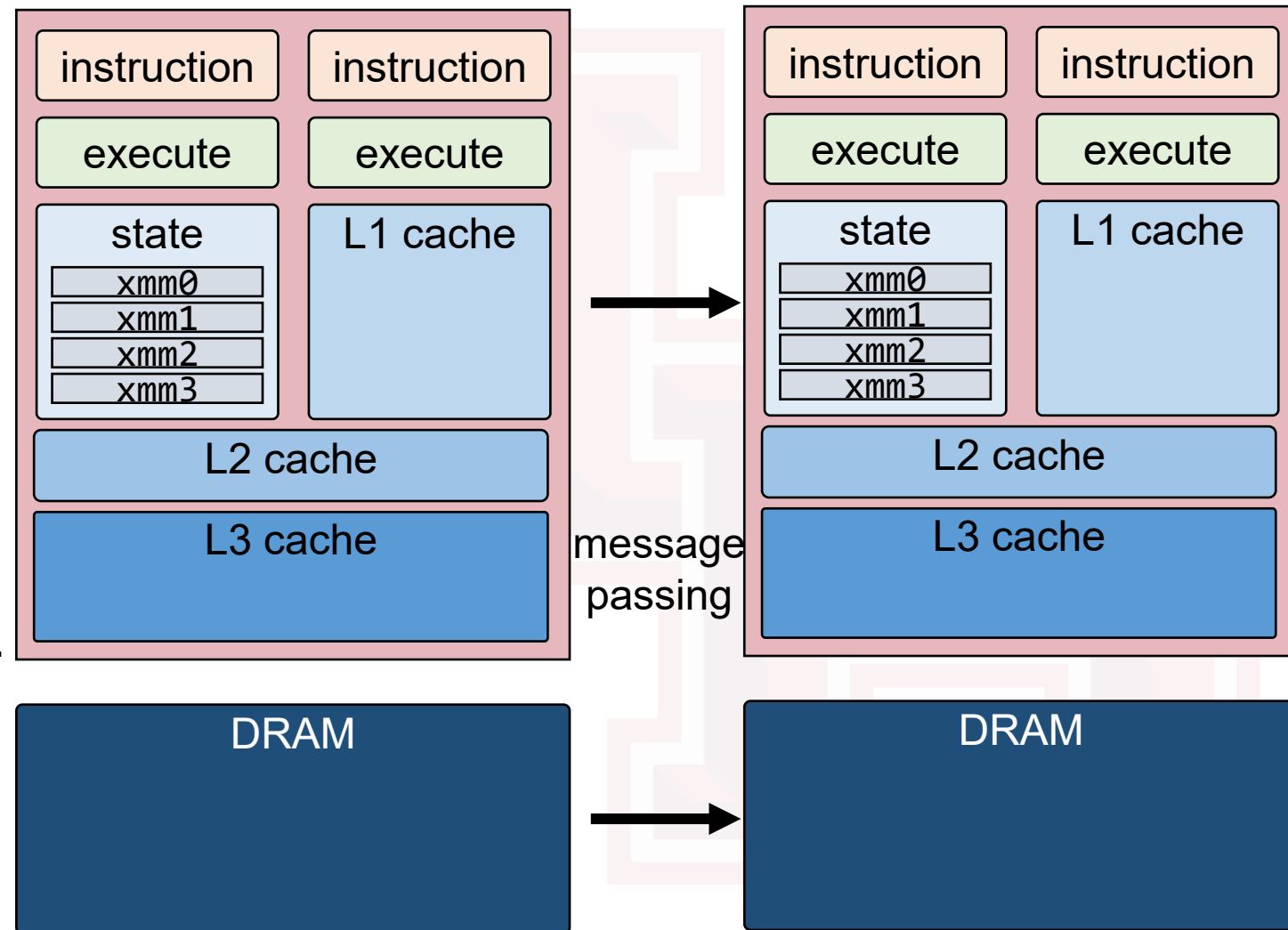
Shared Memory Address Spaces

- Simplifies communication for the programmer
 - All threads can share the same address space
 - Easy to understand synchronization (atomic operations, join(), etc)
- Requires complex control hardware
 - Caches have to be implemented and synchronized (snooping)
 - Hard to scale: chips have to be further apart, requiring more complex control hardware
- **Non-uniform memory access (NUMA)**
 - Fetch time can vary **outside of cache hits/misses**
 - Difficult for the programmer to plan around this



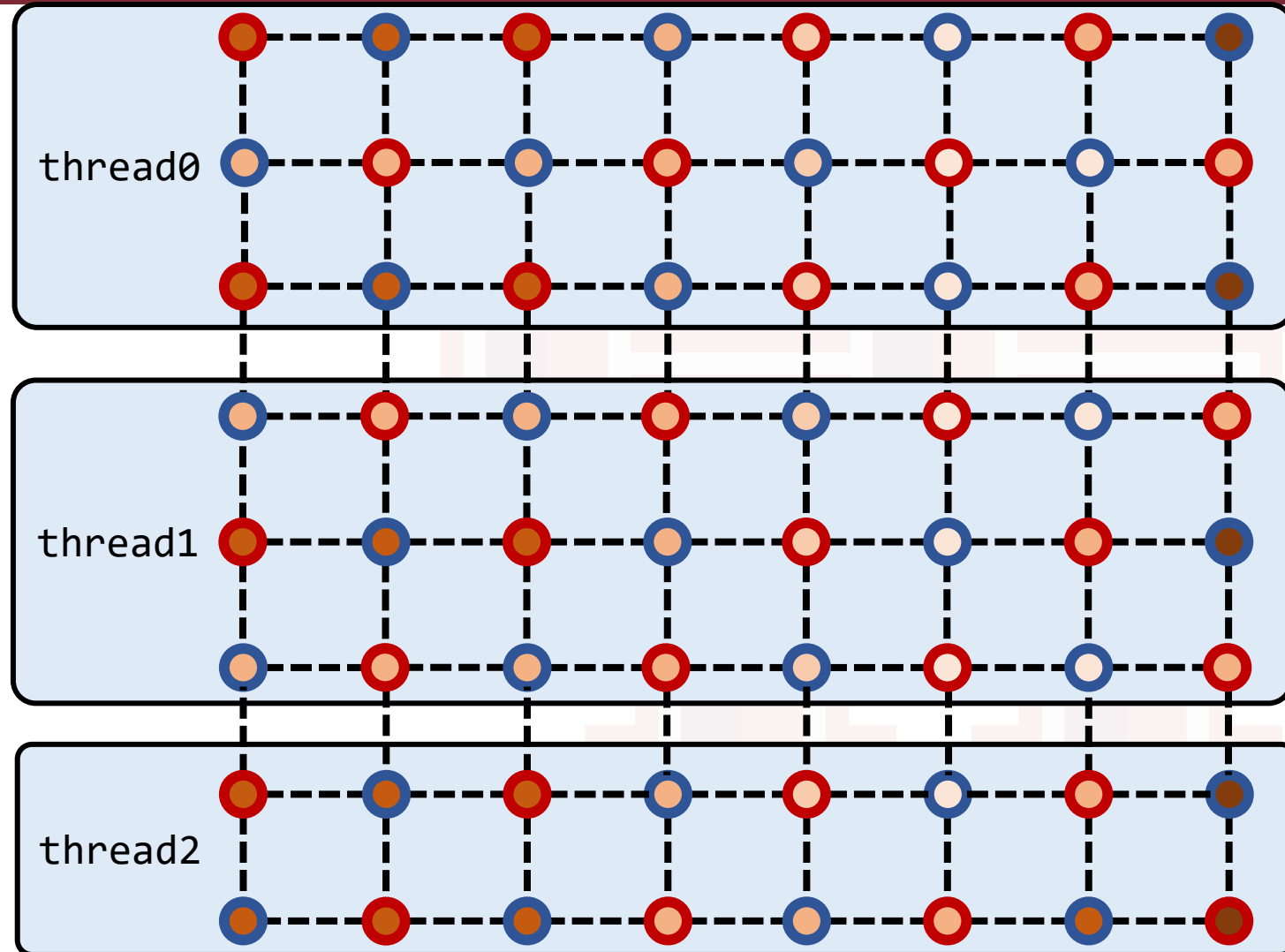
Distributed Memory Address Spaces

- Message-passing model
- Send data from thread0 to thread1:
 - thread0: `send(x[], 1, id)`
 - thread1: `receive(y[], 0, id)`
 - thread1: access `x[]` at location `y[]`
- `send()` specifies data and receiver
- `receive()` specifies local buffer and sender
- Implementation: OpenMPI



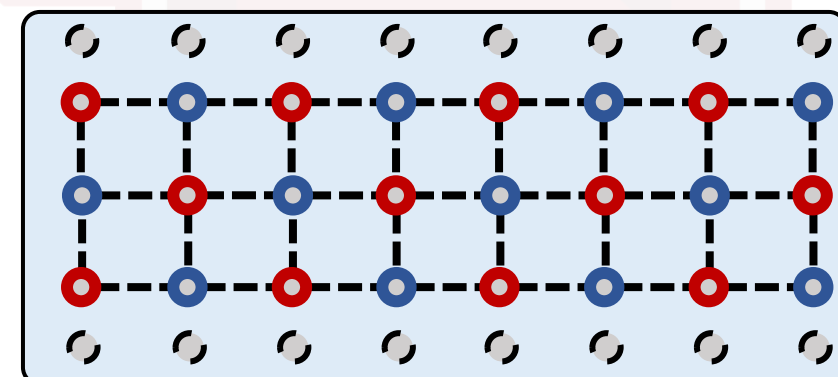
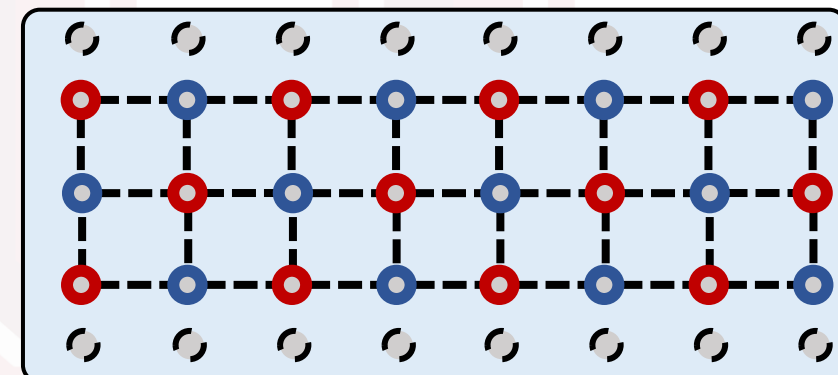
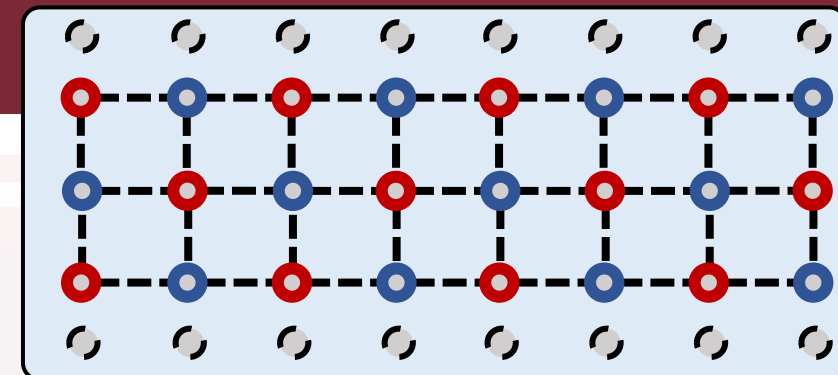
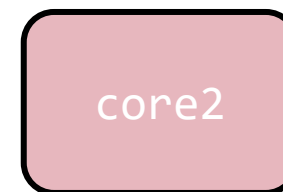
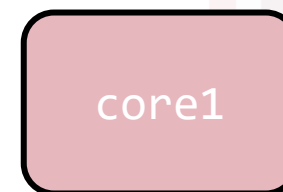
Heat Equation Solver

- Assign work (assume static)
- What data has to be passed?
- How can we arrange this?

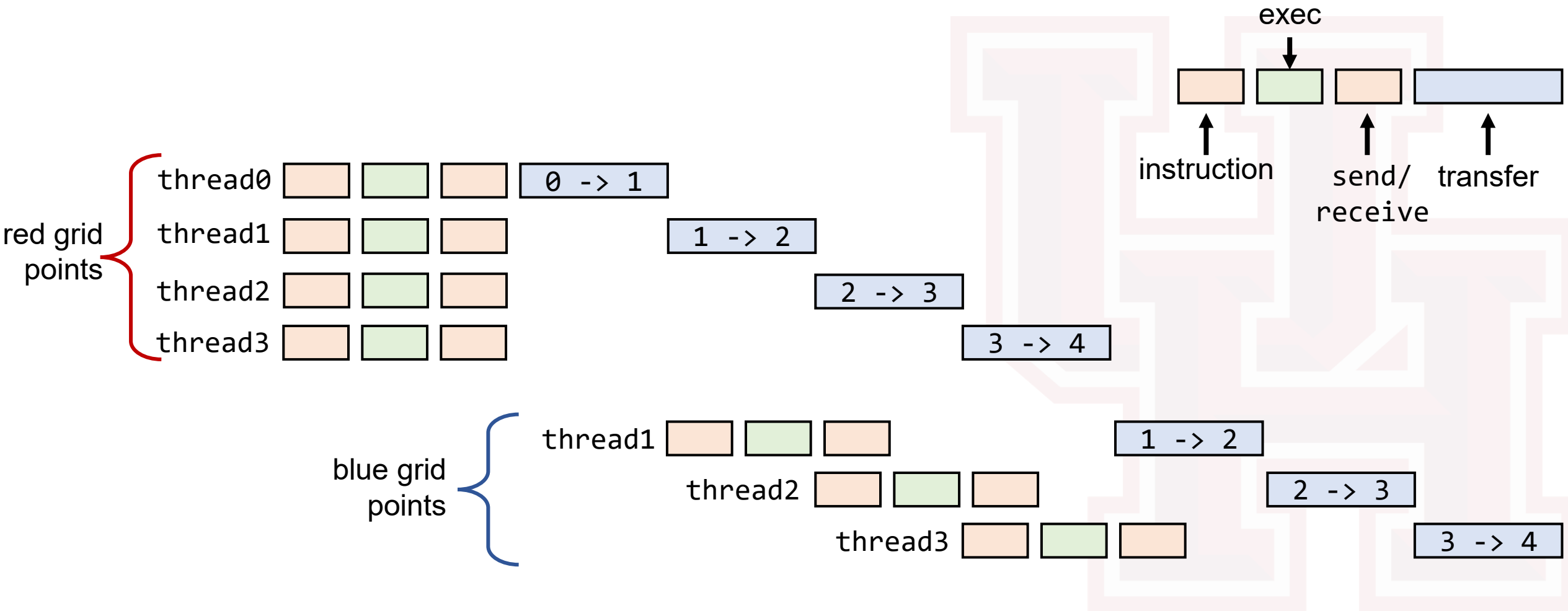


Allocate a “Curtain” or “Ghost Nodes”

- Over-allocate memory for each thread to account for a curtain
- How would you arrange the `send()` and `receive()` instructions?
 - `core_i: send(x[], i+1)`
 - `core_i: receive(y[], i-1)`
 - `core_i: send(x[], i-1)`
 - `core_i: receive(y[], i+1)`
- What are your limitations?
 - `receive()` has to be blocking
 - dependence between threads
- What accounts for overhead?
 - **bandwidth**



Bandwidth Limitations



Minimize Transfers

- Arithmetic Intensity:

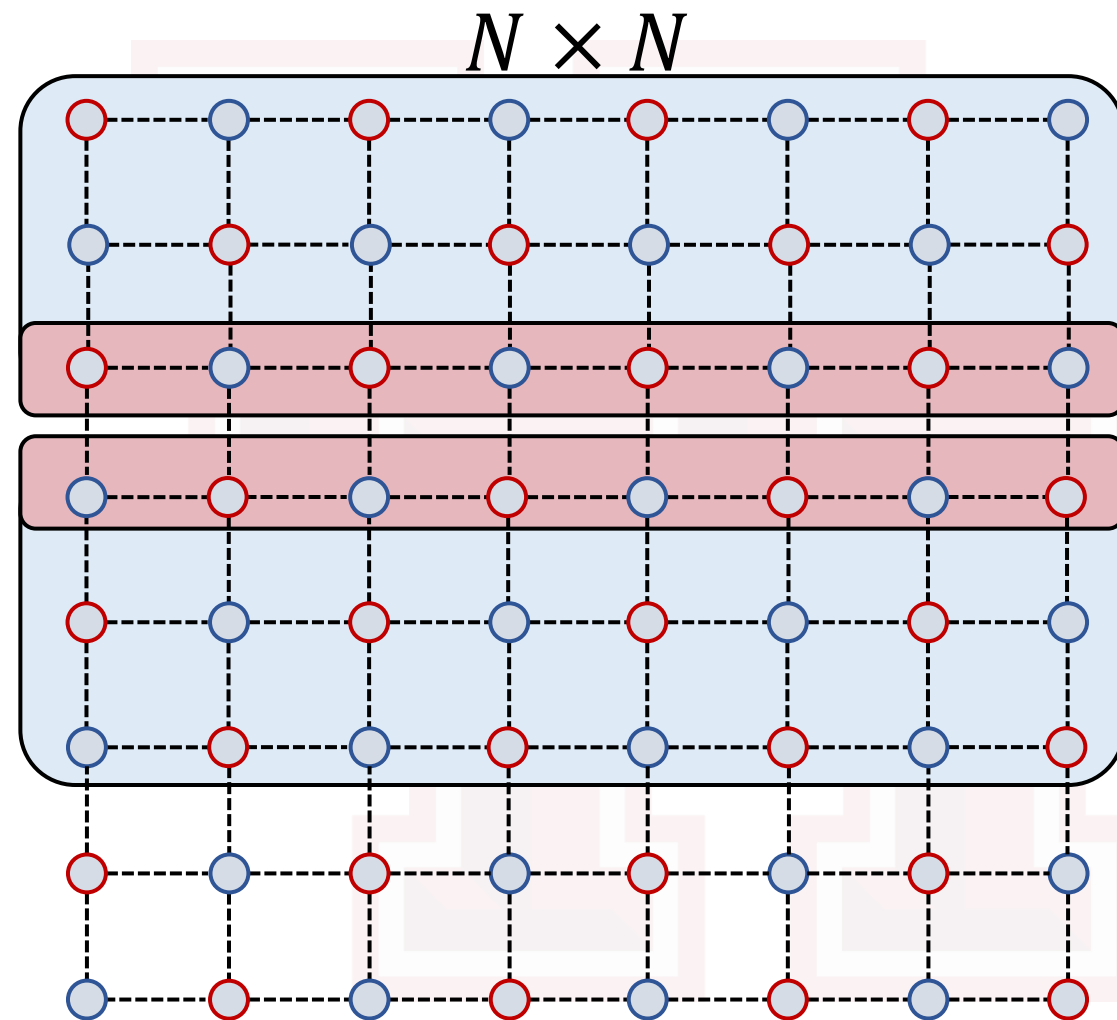
$$A = \frac{C}{B}$$

- C is the computation (ex. FLOPS)
- B is the bandwidth (ex. bytes)

- Blocked assignment (P processors with $N \times \frac{N}{P}$ blocks):

$$A = \frac{N \frac{N}{P}}{2N} = \frac{N^2}{2NP} \approx \frac{N}{P}$$

○



Minimize Transfers

- Arithmetic Intensity:

$$A = \frac{C}{B}$$

- C is the computation (ex. FLOPS)

- B is the bandwidth (ex. bytes)

- Blocked assignment (P processors with $N \times \frac{N}{P}$ blocks):

$$A = \frac{N \frac{N}{P}}{2N} = \frac{N^2}{2NP} \approx \frac{N}{P}$$

- Interleaved assignment $P = N$:

$$A = \frac{N}{2N} \approx \frac{1}{2}$$

